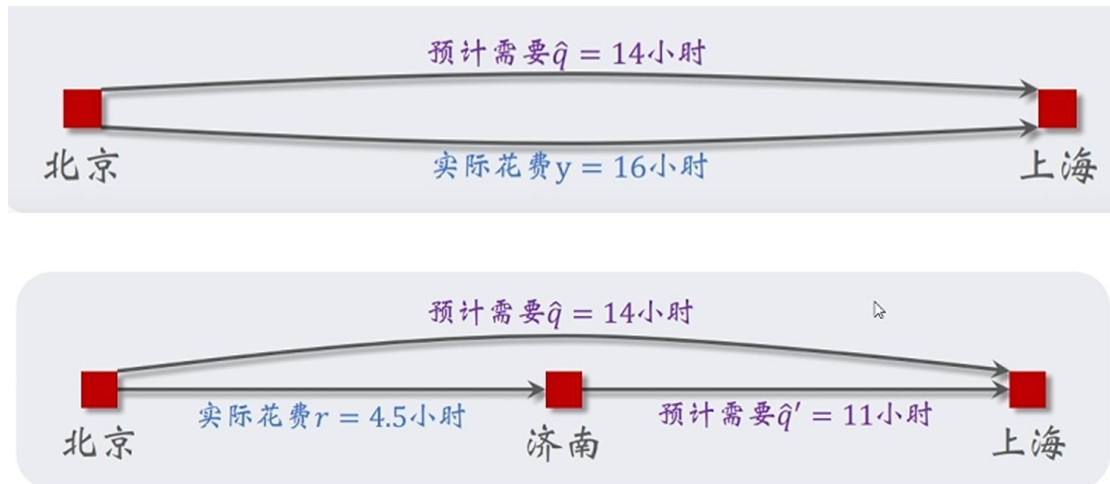


TD 时间差分



Q-Learning 算法

1. 算法原理

Q-Learning的核心在于通过更新**Q值**来估计每个状态-动作对的价值，即在状态 s 下采取动作 a 所能获得的期望奖励。其更新公式为：

$$Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$$

- $Q(s, a)$: 状态 s 下采取动作 a 的当前估计值。
- α : 学习率，控制Q值更新的步长。
- r : 在状态 s 下采取动作 a 后获得的奖励。
- γ : 折扣因子，衡量未来奖励的权重。
- $\max_{a'} Q(s', a')$: 在下一个状态 s' 中估计的最大Q值。

2. 算法流程

1. 初始化Q表格，使每个状态-动作对的Q值较小（如零）。
2. 在状态 s 选择一个动作 a ，根据策略（如 ϵ -greedy策略）来选择动作。
3. 执行动作 a ，观察到新状态 s' 和奖励 r 。
4. 更新 $Q(s, a)$ 的值：
$$Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$$
5. 将状态更新为 s' ，回到步骤2，重复直到收敛。

3. 特点和优缺点

- **优点:** Q-Learning更新时考虑了下一状态的最优动作，因此具有良好的收敛性和有效性。
- **缺点:** 由于采用离策略，Q-Learning在收敛速度上可能比Sarsa稍慢。

1. 即时奖励 r

- 是指智能体在状态 s 下执行动作 a 后，当下（立刻）得到的那部分回报。
- 它通常只和当前一次动作紧密相关，比如执行某个操作得到的分数、赢得或失去的积分等。

2. $Q(s, a)$ 值

- 是指在状态 s 下执行动作 a 所能带来的“长期回报”估计值；也就是说，**不仅包含眼前获得的奖励，还要考虑未来可能获得的累计奖励之和。**
- 其在 Q-Learning 等算法里会不断迭代更新，最终收敛到一个对“长期收益”更准确的估计。

例子：玩吃豆人游戏 (Pac-Man)

• 即时奖励 r

- 假设你在某个位置（状态 s ）决定往右走（动作 a ）。恰好在右边格子上有一个豆子，当你吃掉它之后，你立刻获得 +10 分的奖励，那么这 10 分就是 r ，是**一次性的**即时奖励。

• $Q(s, a)$ 值

- 当你决定在状态 s 往右走时， $Q(s, a)$ 则表示：**如果在这个状态下选择了往右走，不仅会拿到这 10 分，还可能在后续几个动作中获得更多分数或者避免损失，从而得到的长期累计回报的一个估计值。**
- 比如，你可能还能吃到更多豆子，或者成功躲过鬼魂，一路上又赚到其他奖励。这样所有未来可能的奖励都会被折算到 $Q(s, a)$ 中去。

因此，

- r 只代表“当下”立即得到的好处或损失。
- $Q(s, a)$ 则估计了“当下动作 + 后续一系列动作”所带来的总体收益。

在 Q-Learning 更新公式里：

$$Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma \max_{a'} Q(s', a') - Q(s, a)],$$

- r 那一项就是这一次动作得到的**即时奖励**。
- 而 $\max_{a'} Q(s', a')$ 则是对**未来所有可能动作**带来的最高收益的估计，乘以折扣因子 γ 来体现长期回报对当前的影响。
- 由此可见， $Q(s, a)$ 的更新同时依赖了“眼前的收益”（ r ）和“未来的潜在价值”（ $\max_{a'} Q(s', a')$ ）两部分信息。

在 Q-Learning 的更新公式中：

$$Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma \max_{a'} Q(s', a') - Q(s, a)],$$

那一项

$$r + \gamma \max_{a'} Q(s', a') - Q(s, a)$$

之所以要减去 $Q(s, a)$ ，是因为这部分代表了“新目标”与“当前估计值”之间的差，又称为“TD (Temporal Difference) 误差”。简单来说：

- **新目标 (target)** : $r + \gamma \max_{a'} Q(s', a')$ ，即这一步动作得到的即时奖励 + 未来最优动作的折扣回报。
- **当前估计 (old estimate)** : $Q(s, a)$ ，就是我们目前对状态 s 下动作 a 的长期价值估计。

只有**目标与当前估计**之间的差值才告诉我们“还要把当前的 Q 值往上(或往下)修正多少”。用数学表达就是：

$$\underbrace{Q(s, a)}_{\text{旧估计}} + \alpha \cdot \underbrace{\left[\underbrace{r + \gamma \max_{a'} Q(s', a')}_{\text{新目标}} - \underbrace{Q(s, a)}_{\text{旧估计}} \right]}_{\text{TD 误差}}$$

这样更新能使 $Q(s, a)$ **逐步逼近**它在该状态下的**真实长期价值**。如果不减去当前的 $Q(s, a)$ ，就不知道自己和目标还差多少，从而也就无法计算需要“纠正”多少了。通过“差值”乘以学习率 α ，**一点点地**把原来的 $Q(s, a)$ 向目标值调整，这就是增量式学习 (Incremental Learning) 的核心思路。

一、环境说明

1. 状态 (States)

我们用 (x, y) 表示机器人在网格中的位置，共有 4 个位置：

$$S_0 = (0, 0), \quad S_1 = (0, 1), \quad S_2 = (1, 0), \quad S_3 = (1, 1).$$

其中 $(1, 1)$ (即 S_3) 设为**目标位置**。

2. 动作 (Actions)

在每个位置，机器人可以尝试 4 个动作：{Up, Down, Left, Right}。

- 如果动作会走出网格，就保持在原地（为了简单，这里不额外给惩罚）。
- 如果走到目标位置 $(1, 1)$ ，本回合 (Episode) 结束。

3. 奖励 (Reward)

- 当机器人**到达目标** $(1, 1)$ 时，得到 +10 的奖励，回合终止。
- 其他情况下，奖励为 0。

4. Q 表 (Q-Table)

我们用 $Q(s, a)$ 表示在状态 s 下执行动作 a 的价值估计。初始时，所有 $Q(s, a) = 0$ 。

5. 其他超参数

- 学习率 $\alpha = 0.5$
- 折扣因子 $\gamma = 0.9$
- 为了演示，动作的选择过程这里**人为指定**（不额外考虑 ϵ -greedy），主要想看更新公式是如何迭代数值的。

6. Q-Learning 更新公式

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right].$$

其中

- s 为当前状态， a 为当前动作；
- r 为执行动作后得到的即时奖励；
- s' 为执行动作后到达的新状态；
- $\max_{a'} Q(s', a')$ 为在新状态 s' 时所有可选动作的最大 Q 值。
- α 和 γ 分别是**学习率**和**折扣因子**。

二、具体迭代示例

下面我们跑两幕 (Episode) 来看看数值是怎样更新的。

Episode 1

- 初始 Q 表:

所有 $Q(s, a) = 0$ (用 0 表示)。比如:

$$Q((0, 0), \text{Up}) = 0, \quad Q((0, 0), \text{Right}) = 0, \quad \dots$$

以此类推, 对所有状态动作对都初始化为 0。

1. Step 1

- 当前状态: $s = S_0 = (0, 0)$ 。
- 人为指定机器人选动作: Right。
- 执行动作后:
 - 新状态 $s' = S_1 = (0, 1)$;
 - 即时奖励 $r = 0$ (还没到目标)。
- 进行 Q-Learning 更新:
$$Q(S_0, \text{Right}) \leftarrow Q(S_0, \text{Right}) + \alpha [r + \gamma \max_{a'} Q(S_1, a') - Q(S_0, \text{Right})]$$
$$= 0 + 0.5 [0 + 0.9 \times \max_{a'} Q(S_1, a') - 0].$$
 - 由于此时 $Q(S_1, a')$ 仍全是 0, $\max_{a'} Q(S_1, a') = 0$ 。
 - 因此:
$$Q(S_0, \text{Right}) = 0.5 \times (0 + 0.9 \times 0) = 0.$$
 - 更新后依旧是 0。

2. Step 2

- 现在状态: $s = S_1 = (0, 1)$ 。
- 指定选动作: Down。
- 执行动作后:
 - 新状态 $s' = S_3 = (1, 1)$, 这是**目标位置**;
 - 即时奖励 $r = +10$; 回合立即结束。
- Q 更新:
$$Q(S_1, \text{Down}) \leftarrow 0 + 0.5 [10 + 0.9 \times \max_{a'} Q(S_3, a') - 0].$$
 - 到达目标后, 往往视作终止状态, $Q(S_3, a') = 0$ (或该状态不再更新), 所以 $\max_{a'} Q(S_3, a') = 0$ 。
 - 于是:
$$Q(S_1, \text{Down}) = 0.5 \times (10 + 0.9 \times 0) = 0.5 \times 10 = 5.$$
- 回合结束。此时**主要更新**:

$$Q(S_1, \text{Down}) = 5.$$

- **Episode 1 结束后的主要 Q 值**

$$Q(S_0, \text{Right}) = 0, \quad Q(S_1, \text{Down}) = 5,$$

其他都还保持在 0。

Episode 2

继续再来一个回合，让我们看看已有的 Q 值如何影响这次更新。

1. 初始状态: $s = S_0 = (0, 0)$ 。

- 在上一回合结束后: $Q(S_0, \text{Right}) = 0$ 。虽然是 0，但相对其他动作也都是 0，没什么差别，这里依旧指定机器人“Right”移动。
- 新状态 $s' = S_1$ ，即时奖励 $r = 0$ 。
- Q 更新:

$$\begin{aligned} Q(S_0, \text{Right}) &\leftarrow Q(S_0, \text{Right}) + 0.5 \left[0 + 0.9 \max_{a'} Q(S_1, a') - Q(S_0, \text{Right}) \right] \\ &= 0 + 0.5 \left[0 + 0.9 \times \max_{a'} Q(S_1, a') - 0 \right]. \end{aligned}$$

- 现在 S_1 的动作价值中, $\max_{a'} Q(S_1, a') = Q(S_1, \text{Down}) = 5$ 。
- 因而目标项 $0 + 0.9 \times 5 = 4.5$ 。
- TD 误差 $= 4.5 - 0 = 4.5$ 。
- 学习率 $\alpha = 0.5$ ，所以:

$$Q(S_0, \text{Right}) = 0 + 0.5 \times 4.5 = 2.25.$$

2. 下一步: 状态 $s = S_1$

- 由于我们知道 $Q(S_1, \text{Down}) = 5$ ，很可能再次选择 Down 来到终点。
- 执行动作 Down 后: $s' = S_3$ ，即时奖励 $r = +10$ 。

- Q 更新:

$$Q(S_1, \text{Down}) \leftarrow 5 + 0.5 \left[10 + 0.9 \times \max_{a'} Q(S_3, a') - 5 \right].$$

- $\max_{a'} Q(S_3, a') = 0$ ，因此目标项 $= 10$ 。
- TD 误差 $= 10 - 5 = 5$ 。
- 更新后:

$$Q(S_1, \text{Down}) = 5 + 0.5 \times 5 = 5 + 2.5 = 7.5.$$

• Episode 2 结束后主要 Q 值

$$Q(S_0, \text{Right}) = 2.25, \quad Q(S_1, \text{Down}) = 7.5.$$

其他依然是 0。

三、后续迭代与收敛

如果继续跑更多回合，机器人在不同状态下可能尝试各种动作，Q 表会进一步被填满。在多次尝试后，Q-Learning 会逐渐让所有 $Q(s, a)$ 趋近真实的长期价值，也就能学会从每个格子出发时该选哪个动作最快到达目标。

关键在于每一步的更新公式都在做同样的事情：

$$\underbrace{Q(s, a)}_{\text{旧估计}} + \alpha \left[\underbrace{r + \gamma \max_{a'} Q(s', a')}_{\text{新目标}} - \underbrace{Q(s, a)}_{\text{旧估计}} \right].$$

- “新目标” = 当下奖励 + 对后续的价值预测。
- “旧估计” = 之前学到的 $Q(s, a)$ 值。
- 通过两者的差值（TD 误差）来逐步修正旧的 Q 值。

在大量迭代后，当所有状态动作对的 Q 值都与“真实长期回报”一致时，算法就基本收敛了。

小结

1. **即时奖励 r** ：每走一步得到多少分，这里要么 0，要么到目标 +10。
2. **长期价值 $Q(s, a)$** ：不仅包含当下奖励，也要预测未来决策的回报（折扣因子 γ 带入）。
3. **Q-Learning 的每一步更新**：利用“旧估计” + 学习率 $\times$ (“新目标” - “旧估计”) 的方式，反复迭代。
4. **示例中的数字**：
 - 第一回合主要把 $Q(S_1, \text{Down})$ 更新为 5；
 - 第二回合则利用“未来在 S_1 能得到 5”这一点，把 S_0, Right 提升到 2.25，然后又把 S_1, Down 提升到 7.5。
 - 再多来几回合，其它状态和动作也会陆续有非零 Q 值。

代码案例解析：

```
1、 action = np.random.choice(np.flatnonzero(Q_list == Q_list.max()))
```

示例 1：简单 Q 值数组

假设 `Q_list` 是：

```
python

import numpy as np

Q_list = np.array([0.3, 0.5, 0.5, 0.2])
```

 复制  编辑

这代表：

- 动作 0 的 Q 值是 0.3
- 动作 1 的 Q 值是 0.5
- 动作 2 的 Q 值是 0.5
- 动作 3 的 Q 值是 0.2

Step 1：找到最大 Q 值

```
python

Q_list.max()
```

 复制  编辑

计算 `Q_list` 的最大值：

```
python

Q_list.max() # 0.5
```

 复制  编辑

所以最大 Q 值是 0.5。

🚀 示例 2：当 Q 值全部相同时

如果：

```
python

Q_list = np.array([0, 0, 0, 0])
```

📄 复制 🗑 编辑

那么：

```
python

Q_list.max() # 0
Q_list == Q_list.max() # [True, True, True, True]
np.flatnonzero(Q_list == Q_list.max()) # [0, 1, 2, 3]
```

📄 复制 🗑 编辑

随机选择：

```
python

np.random.choice([0, 1, 2, 3])
```

📄 复制 🗑 编辑

最终 `action` 可能是 `0, 1, 2, 3` 中的任何一个。

1. 防止 `argmax` 偏向选择第一个最大 Q 值的索引。

- `np.argmax(Q_list)` 总是选择最左侧的最大值索引（例如 `[0.5, 0.5]` 会选第一个 `0.5`）。
- 但 `np.flatnonzero()` 可以找到所有最大值索引，然后 `np.random.choice()` 随机选择，增强探索能力。

2. 更好的探索行为

- 如果多个动作的 Q 值相同，我们希望随机选择，而不是总是选第一个。
- 这样可以防止策略过早收敛到特定动作，提高 Q-learning 的学习效果。

2、

```
28 ▼ def get_behavior_action(self, obs: int) -> int:
29     """对于这样一种离策略算法，我们只是为探索从目标策略修改了一个epsilon-greedy策略。"""
30     ...
31     采用 ε-greedy 策略，即：
32     以概率 ε 随机选择动作（探索）
33     以概率 1 - ε 选择当前最优动作（利用）
34     ...
35 ▼     if np.random.uniform(0, 1) < self.epsilon:
36         action = np.random.choice(self.action_dim)
37 ▼     else:
38         action = self.get_target_action(obs)
39     return action
```

假设：

- `self.epsilon = 0.1` (10% 概率探索, 90% 概率利用)
- `self.action_dim = 4` (4 个可能的动作)
- `Q_table[obs] = [0.3, 0.5, 0.5, 0.2]` (Q 值)

◆ 可能情况 1 (探索, 10% 概率) :

1. `np.random.uniform(0, 1) = 0.05 < 0.1` → 进入 探索
2. `action = np.random.choice(4)` → 可能得到 0, 1, 2, 3 中的任意一个
3. 返回 `action` (随机)

◆ 可能情况 2 (利用, 90% 概率) :

1. `np.random.uniform(0, 1) = 0.7 > 0.1` → 进入 利用
2. `action = self.get_target_action(obs)` → 选择 Q 值最高的动作
 - `Q_list = [0.3, 0.5, 0.5, 0.2]`
 - `np.flatnonzero(Q_list == 0.5)` → [1, 2]
 - `np.random.choice([1, 2])` → 可能选择 1 或 2
3. 返回 `action` (最优)

3、BOE_iterative_solver()

◆ (1) Bellman Optimality Equation

- 这个方法的核心是基于 贝尔曼最优方程 (Bellman Optimality Equation, BOE) :

$$Q(s, a) = r + \gamma \max_{a'} Q(s', a')$$

- `BOE_iterative_solver` 表示通过迭代逼近贝尔曼最优方程的求解器。

◆ (2) Bootstrapped Off-policy Estimation

- 这个方法是 **Q-learning**, 它是一种 **off-policy** (离策略) 方法, 即:
 - 采样过程中, 使用的是 `ε-greedy` 进行探索
 - 更新 Q 值时, 使用的是 `max(Q(s', a'))` 进行更新 (与当前策略无关)

4、`TD_target = reward + (1 - float(done)) * self.gamma * self.Q_table[next_obs, :].max()`

- 如果 `done = True` (终止状态) :
 - `TD_target = reward`
- 如果 `done = False` (未终止) :
 - `TD_target = reward + γ * max(Q(next_state, a'))`
 - `max(Q(next_state, a'))` 代表下一状态的最大 Q 值
 - `γ` (gamma, 折扣因子) 决定未来奖励的影响程度

示例

假设：

```
python

reward = 1
gamma = 0.9
next_obs = 1
Q_table[1, :] = [0.5, 0.6, 0.7, 0.8] # 状态 1 的 Q 值
done = False
```

计算 TD_target

```
python

TD_target = 1 + (1 - 0) * 0.9 * max([0.5, 0.6, 0.7, 0.8])
TD_target = 1 + 0.9 * 0.8
TD_target = 1.72
```

更新 Q 值：

```
python

self.Q_table[obs, action] -= self.lr * (current_Q - TD_target)
```

使用 **TD 误差 (Temporal Difference Error)** 更新 Q 值：

$$\Delta Q = \alpha \cdot (TD_target - current_Q)$$
$$Q(s, a) \leftarrow Q(s, a) + \alpha \cdot (r + \gamma \max_{a'} Q(s', a') - Q(s, a))$$

其中：

- α (学习率 `self.lr`) : 控制学习的步长
- `TD_error = TD_target - current_Q` : 时间差分误差

示例

假设：

```
python

lr = 0.1 # 学习率
current_Q = 0.3 # Q(0,2) = 0.3
TD_target = 1.72 # 计算出的目标值
```

计算 Q 值更新

```
python

Q_new = 0.3 + 0.1 * (1.72 - 0.3)
Q_new = 0.3 + 0.1 * 1.42
Q_new = 0.3 + 0.142
Q_new = 0.442
```

python

复制 编辑

```
Q_table[0,2] = 0.442 # Q 值更新
```

完整代码：

假设：

python

复制 编辑

```
self.Q_table = np.array([[0.1, 0.2, 0.3, 0.4], # 状态 0
                          [0.5, 0.6, 0.7, 0.8]]) # 状态 1

obs = 0 # 当前状态
action = 2 # 选择动作 2
reward = 1 # 立即奖励
next_obs = 1 # 进入状态 1
done = False # 不是终止状态
gamma = 0.9 # 折扣因子
lr = 0.1 # 学习率
```

执行 BOE_iterative_solver :

python

复制 编辑

```
current_Q = self.Q_table[obs, action] # 0.3
TD_target = reward + (1 - float(done)) * self.gamma * self.Q_table[next_obs, :].max()
TD_target = 1 + (1 - 0) * 0.9 * max([0.5, 0.6, 0.7, 0.8])
TD_target = 1 + 0.9 * 0.8
TD_target = 1.72

self.Q_table[obs, action] -= self.lr * (current_Q - TD_target)
Q_new = 0.3 + 0.1 * (1.72 - 0.3) = 0.442
```

最终 Q_table 更新：

python

复制 编辑

```
Q_table[0,2] = 0.442
```

5、Reward 是怎么获得的？（Reward 不是 Q 表算出来的，是你自己定义的，也可以认为 reward 是环境 env 根据当前状态和动作计算的）

在 Q-learning 过程中，reward（奖励）是由环境 env 给出的，每次智能体执行一个动作，环境会返回一个 reward，这个值决定了当前动作的好坏。

```
next_obs, reward, terminated, truncated, _ = self.env.step(action)
```

然后 reward 被传递到 BOE_iterative_solver() 进行 Q 值更新：

```
def BOE_iterative_solver(self, obs: int, action: int, reward: float, next_obs: int, done: bool) -> None:
```

📌 代码逻辑总结

步骤	作用
获取当前 Q 值	<code>current_Q = self.Q_table[obs, action]</code>
计算 TD 目标	<code>TD_target = reward + γ * max(Q(next_obs, a'))</code>
更新 Q 值	<code>Q(s,a) = Q(s,a) + α * (TD_target - current_Q)</code>

💡 核心思想

- Q-learning 采用 `max(Q(s', a'))`，而 Sarsa 采用 `Q(s', a')`。
- 终止状态 `done = True` 时，`TD_target = reward`（无未来 Q 值）。

🚀 这样，强化学习代理通过多次交互和 Q 值更新，最终学会最优策略！

事件	奖励值
普通移动	-1
掉下悬崖（回到起点）	-100
到达终点	0

举例：

🏆 奖励规则

- 正常行走： `reward = -1`（每一步 -1）
- 掉下悬崖（Cliff）： `reward = -100`
- 到达终点（Goal）： `reward = +100`

🚀 计算示例

状态 (obs)	选择 action	next_obs (下一状态)	reward
(3,0) (起点)	右 (→)	(3,1)	-1
(3,1)	右 (→)	(3,2)	-1
(3,9)	右 (→)	(3,10)	-1
(3,10)	右 (→)	(3,11) (终点)	+100
(3,5)	下 (↓)	掉下悬崖 (Reset 到 (3,0))	-100

5、train_episode()和 train(self) -> None 的区别

- `train_episode(self, is_render: bool = False) -> float`
 - 作用：执行一个完整的训练回合（Episode），更新 Q-table，并返回该回合获得的总奖励。
 - 用途：在 `train()` 方法中被调用，每次执行一轮环境交互训练。
- `train(self) -> None`

- **作用：**管理整个 Q-learning 训练过程，运行多个 **Episode**，并在训练完成后测试智能体的表现。
- **用途：**负责训练多个 **Episode**，并在训练过程中间歇渲染环境，最后测试训练后的策略。

总结：

步骤	作用
环境重置	<code>obs, _ = self.env.reset()</code>
循环训练	<code>while True:</code>
选择动作	<code>action = self.agent.get_behavior_action(obs)</code>
执行动作	<code>next_state, reward, terminated, truncated, _ = self.env.step(action)</code>
判断是否终止	<code>done = terminated or truncated</code>
累积奖励	<code>total_reward += reward</code>
更新 Q 值	<code>self.agent.BOE_iterative_solver(obs, action, reward, next_state, done)</code>
状态更新	<code>obs = next_state</code>
渲染环境	<code>self.env.render()</code> （可选）
终止循环	<code>if done: break</code>
返回总奖励	<code>return total_reward</code>

